

SIH26131

CROP DISEASE & PEST DETECTION

Master Software Development & Implementation Report

Software-first • Hardware-ready • AI + IoT simulation • GIS • Expert validation • Early warning

This report is the master blueprint for building the hackathon prototype. No physical hardware is required for the prototype: weather stations, soil sensors, leaf-wetness sensors and pest traps are represented by a Digital Farm Simulator using the same telemetry contracts planned for future real devices.

Core capability	Prototype implementation
AI vision	Crop disease / pest identification from images
Severity	Affected-area estimation or validated severity model
Digital hardware	Realistic simulated telemetry and failure scenarios
Risk intelligence	AI + environment + pest + history fusion
GIS	Zone-level risk and hotspot mapping
Human validation	Expert/extension-worker review workflow
Advisory	Source-controlled, multilingual IPM guidance
Monitoring	Alerts, rechecks, outcomes and feedback

Version 1.0 • 27 August 2026

1. Executive Summary

Build a Crop Health Intelligence Platform, not merely a disease-classification app. The platform should combine crop images, environmental telemetry, pest observations, crop context, historical cases and expert validation to support early detection and management.

Core lifecycle: Capture → Validate → Detect → Quantify → Combine evidence → Forecast risk → Map hotspot → Validate → Advise → Alert → Recheck → Learn.

The hackathon implementation is software-first. The Digital Farm Simulator generates realistic sensor and pest-trap data, but the system is architected so future ESP32 or other field devices can replace the simulator without changing the business logic.

2. Problem Understanding

The practical problem is that crop diseases and pests can be identified late, data is often fragmented, and a disease prediction alone does not tell a farmer where risk is increasing or what to do next. A useful solution must turn multiple observations into a decision-support workflow.

Challenge	Our response
Late visible detection	Early-warning risk trend using environmental + pest + history signals
Image uncertainty	Quality gate, confidence and expert escalation
Fragmented information	Single farm/zone intelligence layer
Poor spatial visibility	GIS zone and hotspot mapping
Generic advice	Controlled knowledge base with validated recommendations
No follow-up	Recheck and outcome tracking
Hardware dependency	Digital simulator with hardware-compatible interfaces

3. Product Goal

Primary goal: provide earlier, explainable and actionable crop-health intelligence while keeping the platform ready for future physical devices.

4. Goals & Success Criteria

ID	Goal	Definition of success
G01	Disease detection	Supported image returns diagnosis + confidence
G02	Pest detection	Supported pest image returns class/count/boxes
G03	Image safety	Bad input triggers retake/uncertain state
G04	Severity	Affected area % or validated severity band
G05	Environmental intelligence	Raw readings converted to documented features
G06	Pest-trap intelligence	Counts/trends and spikes detected
G07	Early warning	Risk trend and threshold events visible
G08	GIS	Zone-level risk and hotspots displayed
G09	Expert validation	Cases can be confirmed/corrected
G10	Advisory	Approved guidance selected by context
G11	Multilingual	Core farmer output localized
G12	Follow-up	Recheck changes case status
G13	Learning	Feedback stored for evaluation
G14	Hardware-ready	Real device can use same telemetry contract
G15	Reliability	Full demo runs without physical hardware

5. Scope Strategy

MVP: one primary crop, 4–8 disease/healthy classes, optional 1–3 pest classes if data is credible, 6–12 farm zones, simulated weather/soil/pest telemetry, real crop images, one expert-review flow and English + Tamil UI.

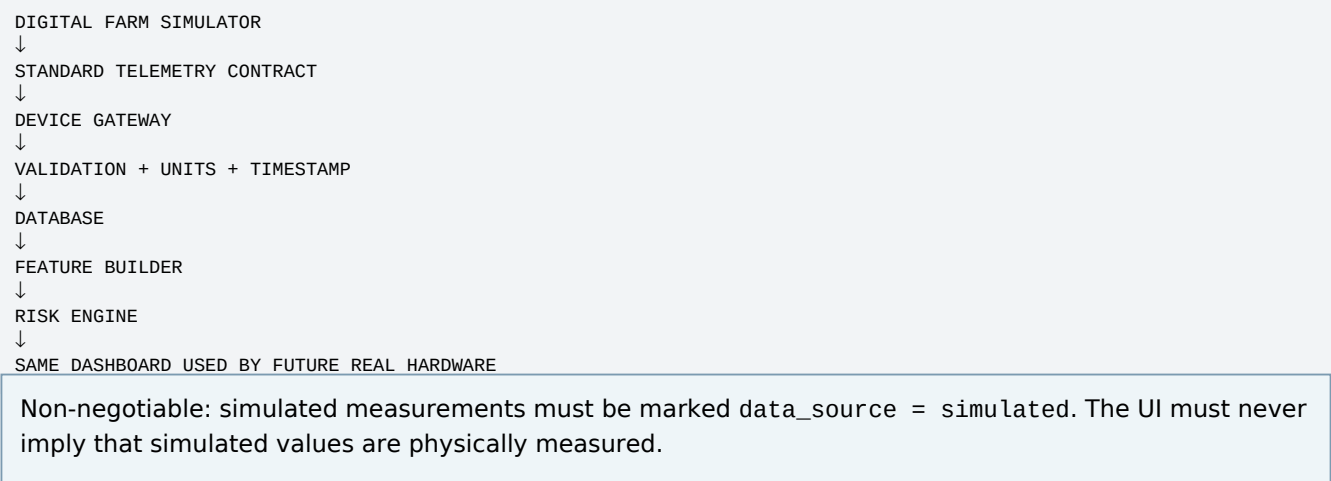
Expansion: additional crops, real weather API, real sensors, better forecasting, edge inference, trap-image recognition and larger regional GIS.

Rule: never claim universal diagnosis. Clearly define supported crops/classes and the limits of the prototype.

6. Master Architecture



7. Hardware Simulation Principle



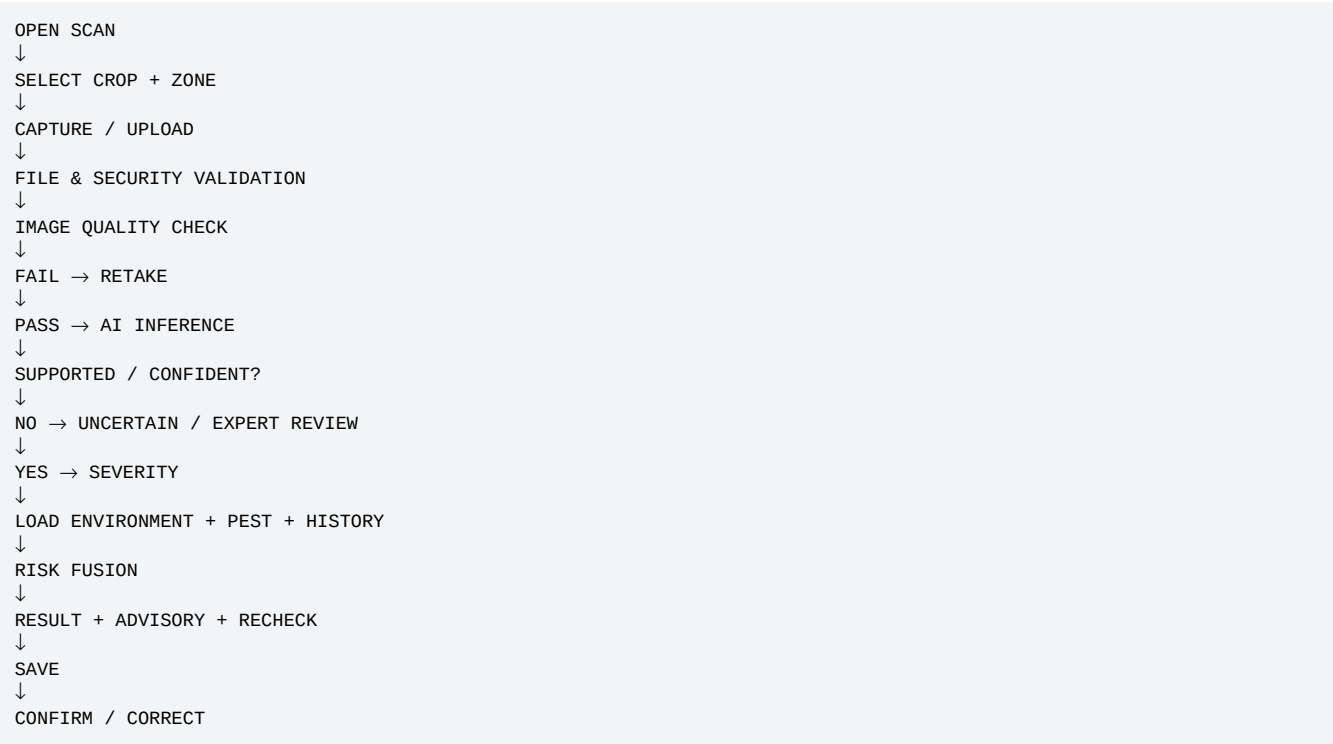
8. Major Software Components

Component	Responsibility
Farmer App	Scan, results, risk, alerts, history, language
Officer/Expert Dashboard	Cases, map, hotspots, validation, analytics
API Gateway	Auth, routing, validation, rate limiting, errors
Image Quality	Blur/brightness/resolution/leaf checks
AI Vision	Disease/pest classification and detection
Severity	Segmentation or validated severity model
Device Gateway	Simulator and future device telemetry
Weather Adapter	Simulated now; external weather later
Feature Engine	Time windows, trends and derived variables
Risk Engine	Evidence fusion and early-warning logic
GIS Engine	Zone risk and hotspot visualization
Advisory Engine	Validated IPM knowledge selection
Alert Engine	Risk, sensor, pest and recheck alerts
Feedback/Analytics	Confirmation, corrections and KPIs

9. Farmer Application Blueprint

Screen	Function
Onboarding	Role, language, farm, crop
Home	Health summary, current risk, alert and device status
Scan	Camera/upload + crop/zone selection
Analysis	Quality and inference progress
Result	Diagnosis, confidence, severity, evidence, risk, action
Farm Map	Zone colors, hotspots, cases and sensor status
History	Scans, risk trend, severity trend
Alerts	Active, acknowledged and resolved alerts
Settings	Language, notifications and device settings

10. Farmer Scan Flowchart



11. Result Design

Diagnosis: Early Blight Confidence: 91% Severity: 14% Environmental Risk: High Pest Trend: Rising Combined Risk: 86/100 Evidence: lesion pattern + recent wet conditions Status: AI prediction – expert confirmation recommended

12. AI Development Pipeline

- 1 Freeze crop and class taxonomy.
- 2 Collect and clean data; document sources and labels.
- 3 Prevent duplicate/same-plant leakage across train and test.
- 4 Create train/validation/test plus separate field-style test data.
- 5 Train a transfer-learning baseline.
- 6 Evaluate macro precision/recall/F1, per-class recall and confusion matrix.
- 7 Add image-quality checks and confidence calibration.
- 8 Add pest detection only if suitable annotations exist.
- 9 Add segmentation/severity if time and data support it.
- 10 Export a stable inference model and record model/dataset versions.

13. AI Service Contract

Input: image, crop, optional crop_stage Output: scan_id model_version diagnosis confidence probabilities[]
pest_boxes[] severity_percent inference_time_ms

The AI service should not generate unrestricted agricultural treatment advice. Prediction, explanation and advisory are separate layers.

14. Image Quality System

```
IMAGE
↓
FORMAT / SIZE
↓
BLUR
↓
BRIGHTNESS / CONTRAST
↓
RESOLUTION
↓
LEAF VISIBILITY
↓
SUPPORTED CROP
↓
QUALITY OK?
↓
NO → RETAKE INSTRUCTIONS
↓
YES → AI
```

Use understandable retake instructions: move closer, improve lighting, keep leaf in frame, hold camera steady.

15. Severity Engine

Preferred method: segment the leaf and diseased region, then estimate affected area.

```
leaf_pixels = pixels in relevant leaf mask
disease_pixels = diseased pixels inside leaf
severity_percent =
disease_pixels / leaf_pixels × 100
```

Validate thresholds with annotated data. If segmentation is not feasible, use a validated severity classifier and label the method honestly.

16. Pest Detection Engine

For image-based pest detection, return pest class, confidence and bounding boxes/count where the dataset supports it. For simulated traps, ingest pest counts as a separate evidence stream.

17. Digital Farm Simulator

Virtual device	Data	Scenario controls
Weather station	Temperature, humidity, rainfall, wind	Normal / rain / humid event
Soil node	Soil moisture, soil temperature	Drying / wetting
Leaf wetness	Wetness level/duration	Rain/dew event
Pest trap	Pest type + count/time	Stable / rising / spike
Gateway	Online / offline / reconnect	Failure simulation

Simulator scenarios must be deterministic/replayable for demos so the same story can be reproduced.

18. Telemetry Contract

```
{ device_id: "SIM-WEATHER-01", zone_id: "ZONE-B", timestamp: "...", temperature_c: 28.4, humidity_pct: 84, soil_moisture_pct: 61, leaf_wetness: 0.72, pest_count: 12, data_source: "simulated" }
```

Future ESP32/field devices should send the same logical schema. Add `firmware_version`, calibration metadata and device health when physical devices are introduced.

19. Environmental Feature Engine

Derive documented features such as 6/12/24-hour humidity averages, maximum humidity, temperature range, recent rainfall, leaf-wetness duration, soil-moisture trend and pest-count growth rate.

20. Risk Engine

The risk engine combines multiple evidence streams. It should produce a score, band, trend and reasons—not just a number.

```
risk = clamp( w1*AI_probability + w2*environment_risk + w3*pest_risk + w4*severity + w5*history, 0, 1)
```

Weights are configuration. Calibrate them with field data when available. Without sufficient field data, describe them as prototype heuristics.

21. Early Warning

```
CURRENT RISK
↓
COMPARE WITH PRIOR 6/12/24 HOUR VALUES
↓
RISK RISING RAPIDLY?
↓
YES → EARLY WARNING
↓
COMBINE WITH AI / PEST EVIDENCE
↓
HIGH PRIORITY ZONE
↓
CREATE ALERT + RECHECK
```

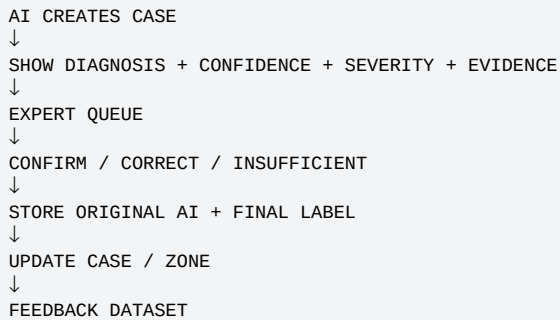
22. GIS & Hotspot Engine

Divide the demo farm into zones. Every image, device and pest trap is attached to a zone. Display current risk, recent cases, severity trend and sensor status.



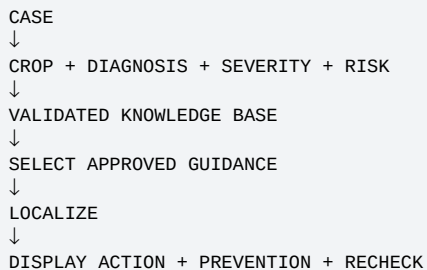
A hotspot is a concentration of recent risk/cases/severity/pest evidence, not proof of disease spread. Label it as a surveillance hotspot.

23. Expert Validation Workflow



Store reviewer role, timestamp, original prediction, final label and comments.

24. Advisory / IPM Engine



Do not fabricate chemical dosages, mixtures, waiting periods or legal instructions. Such information requires authoritative validation.

25. Multilingual System

Keep internal results language-neutral. Translate labels, explanations and approved advisory content at the presentation layer. Start with English + Tamil, then expand.

26. Alert Engine

Trigger	Alert
High combined risk	Priority inspection
Rapid risk increase	Early warning
Pest spike	Pest surveillance
New confirmed case	Zone monitoring
Sensor offline	Device warning
Low-confidence scan	Retake/expert review
Recheck due	Follow-up reminder

Every alert has zone, reason, timestamp, severity and state.

27. Follow-up Flow

```
DETECTION
↓
RECOMMENDED ACTION
↓
RECHECK WINDOW
↓
NEW IMAGE / OBSERVATION
↓
NEW SEVERITY + RISK
↓
COMPARE WITH PREVIOUS
↓
IMPROVING / STABLE / WORSENING
↓
UPDATE CASE
```

28. Database Blueprint

Entity	Core fields
users	id, role, language
farms	id, owner, crop, area
zones	id, farm, location, crop_stage
devices	id, zone, type, source, status, last_seen
sensor_readings	device, zone, timestamp, values, source
images	id, zone, timestamp, storage_key, quality
predictions	image, model_version, class, confidence
severity_results	prediction, area_percent, band, method
risk_events	zone, timestamp, score, band, reasons
expert_reviews	case, reviewer, decision, corrected_label
recommendations	diagnosis, severity, action, source, version
alerts	zone, type, severity, status
feedback	prediction, label, note
models	name, version, dataset, metrics

29. Backend API

API	Purpose
POST /api/v1/scans	Create scan / workflow
POST /api/v1/images/quality	Image quality
GET /api/v1/scans/{id}	Result
POST /api/v1/telemetry	Simulator / device ingestion
GET /api/v1/devices/{id}/latest	Latest telemetry
GET /api/v1/zones/{id}/risk	Zone risk
GET /api/v1/farms/{id}/map	GIS data
POST /api/v1/expert/reviews	Expert decision
POST /api/v1/feedback	Farmer feedback
GET /api/v1/alerts	Alerts
POST /api/v1/simulator/scenario	Run simulation
GET /api/v1/models	Model metadata

30. Frontend / Backend Technology

Layer	Recommended
Mobile/Web	Flutter or React/React Native
Backend	Python FastAPI
ML	PyTorch
Database	PostgreSQL
Realtime/telemetry	MQTT or HTTPS
Cache/queue	Redis optional
GIS	Web/mobile map layer; grid-map acceptable for MVP
Deployment	Docker + local/cloud
Version control	Git

31. Repository Structure

```
crop-health/ ■■■ frontend/ ■■■ backend/ ■ ■■■ api/ ■ ■■■ core/ ■ ■■■ models/ ■ ■■■ schemas/ ■ ■■■ services/ ■  
■■■ tests/ ■■■ ml/ ■ ■■■ datasets/ ■ ■■■ training/ ■ ■■■ inference/ ■ ■■■ detection/ ■ ■■■ evaluation/ ■■■  
simulator/ ■■■ iot/ ■■■ docs/ ■■■ README.md
```

32. Security

- Authentication and role-based access.
- Separate future device credentials from user credentials.
- Validate file type and size.
- HTTPS in network deployments.
- Secrets only in environment/configuration.
- Rate-limit expensive inference endpoints.
- Audit important actions.
- Do not store unnecessary personal data.

33. Error & Degraded Mode

```
ERROR
↓
CLASSIFY: IMAGE / AI / NETWORK / DATABASE / SENSOR
↓
RETRY IF SAFE
↓
FALLBACK IF AVAILABLE
↓
SHOW USER ACTION
↓
LOG
↓
CONTINUE DEGRADED MODE
```

Failure	Response
Blurry image	Retake guidance
Unsupported crop	Do not diagnose
Low confidence	Uncertain + expert review
AI unavailable	Cached/local demo fallback
Sensor offline	Stale-data indicator
Network loss	Local/cache where implemented
Invalid telemetry	Reject + log

34. Demo Reliability

Prepare a fallback dataset and pre-tested images. A live simulator should be available, but the demonstration must not depend on an external weather API or physical device.

Never hide fallback behavior from judges. Clearly identify simulated or cached demo data.

35. Testing Strategy

Layer	Tests
Unit	Risk formula, severity, validation, recommendation rules
API	Auth, upload, telemetry, error contracts
AI	Held-out + field-style test
Integration	App→API→AI→DB; simulator→gateway→risk
E2E	Scan→risk→advisory→expert→follow-up
Resilience	Network/model/sensor failures
Security	Uploads, auth, secrets, rate limits

36. AI Evaluation

- Classification: macro precision/recall/F1, per-class recall, confusion matrix.
- Detection: precision, recall and mAP where applicable.
- Segmentation: IoU/Dice where applicable.
- Risk: precision/recall by risk band, calibration and false-alert rate.
- Latency: model inference and end-to-end response time.

37. Development Roadmap

Phase	Build	Output
0	Scope + architecture	Crop/classes/contracts frozen
1	Dataset	Clean splits + field-style test
2	AI baseline	Model + metrics
3	Backend	API + DB + auth
4	Farmer scan	Image→result
5	Quality/severity	Reliable input + damage
6	Simulator	Time-series telemetry
7	Risk engine	Multimodal risk
8	GIS	Zones + hotspots
9	Expert	Review/correction
10	Advisory	Approved guidance
11	Alerts/follow-up	Monitoring loop
12	Language	English/Tamil
13	QA	E2E + failure tests
14	Hardening	Fallback + reproducible demo
15	Pitch	3-minute story + metrics

38. Team Allocation

Role	Deliverables
AI/ML	Dataset, models, evaluation, inference
Backend	API, DB, auth, risk services
Simulator/IoT	Telemetry simulator and future device protocol
Frontend	Farmer app, dashboard, GIS, alerts
GIS/Data	Zones, trends, hotspots
Agronomy/Product	Knowledge base, UX, expert workflow
QA/Integration	E2E tests, deployment, demo

39. Winning Demonstration

- 1 Start with a six-zone demo farm; all zones are low risk.
- 2 Open Digital Farm Simulator and trigger a rain/humidity event plus pest-count rise in one zone.
- 3 Show the time-series data and risk beginning to rise.
- 4 Capture a prepared field-style crop image from that zone.
- 5 Show image quality, AI diagnosis and confidence.
- 6 Show severity and evidence.
- 7 Combine AI + environmental + pest + history into a high risk score.
- 8 Show the GIS hotspot.
- 9 Send the case to expert review and confirm it.
- 10 Show validated multilingual advisory.
- 11 Create a recheck.
- 12 Simulate the follow-up and show improving/worsening trend.

40. Three-Minute Demo Flowchart

```
0:00 PROBLEM
↓
0:20 DIGITAL FARM + TELEMETRY
↓
0:45 AI SCAN
↓
1:15 SEVERITY + EXPLANATION
↓
1:35 MULTIMODAL RISK
↓
1:55 GIS HOTSPOT
↓
2:15 EXPERT VALIDATION
↓
2:30 ADVISORY + ALERT
↓
2:45 FOLLOW-UP / FEEDBACK
↓
3:00 IMPACT + HARDWARE-READY FUTURE
```

41. Judge Questions

Question	Answer
Why not only AI?	Because early management needs visual + environmental + pest + spatial evidence.
Where is hardware?	The prototype is software-first; a device abstraction layer uses the same contract future hardware will use.
Is simulation fake?	It is explicitly simulated telemetry used to validate the software pipeline, not represented as physical measurement.
How is AI trusted?	Quality gate, confidence, evidence, expert validation and measurable evaluation.
Can it scale?	Models, crops and devices are modular behind stable interfaces.
What happens next?	Connect devices, collect field labels, calibrate risk and validate field impact.

42. Definition of Done — MVP

- One crop and defined classes work end-to-end.
- Real crop image scan works.
- Poor image handling works.
- AI result is stored with model version.
- Digital simulator produces realistic telemetry.
- Risk engine combines AI + simulated context.
- Zone map/hotspot works.
- Expert review works.
- Approved advisory appears.
- Alerts and follow-up work.
- Full demo works without physical hardware.

43. Definition of Done — Winning Version

- Field-style AI validation is reported.
- Severity is measurable and visualized.
- Risk trend is explainable.
- Hotspot is linked to cases.
- Expert confirmation is recorded.
- Multilingual output works.
- Simulator has repeatable scenarios and failures.
- Hardware interface is documented.
- Metrics and limitations are transparent.
- No simulated measurement is presented as physical data.

44. Final Master Flowchart

```
FARM SETUP
↓
IMAGE + DIGITAL/REAL DEVICE DATA
↓
VALIDATE
↓
AI VISION
↓
DISEASE / PEST + SEVERITY
↓
ENVIRONMENT + PEST + HISTORY
↓
MULTIMODAL RISK
↓
EARLY WARNING
↓
GIS HOTSPOT
↓
EXPERT VALIDATION
↓
SAFE MULTILINGUAL ADVISORY
↓
ALERT + FOLLOW-UP
↓
OUTCOME / FEEDBACK
↓
ANALYTICS + FUTURE MODEL IMPROVEMENT
```

45. Final Product Statement

“A software-first, hardware-ready crop health intelligence platform that combines AI crop-image analysis, environmental and pest observations, farm history and geospatial intelligence to detect problems early, forecast risk, identify hotspots, involve experts when needed, deliver validated multilingual advisories, and continuously monitor outcomes.”

46. Implementation Rules

- Start with one crop and a defensible class set.
- Build the end-to-end path before adding features.
- Keep simulated and real device interfaces identical.
- Measure AI performance; never invent accuracy.
- Treat risk as decision support, not proof of disease.
- Keep agricultural advice source-controlled and expert-reviewed.
- Label simulated/cached data clearly.
- Build a reliable demo before polishing advanced features.

This report is the master software plan. Crop-specific thresholds, disease biology and real-world treatment recommendations must be validated with authoritative agricultural sources and qualified experts before deployment.